

TECNOLOGÍAS DE LA INFORMACIÓN Desarrollo de Sistemas Informáticos



Profesor:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: Mauro Vinicio Pomagualli Yambay.

PARALELO: "A"

SEMESTRE: Quinto

ACTIVIDAD: Desarrollo de una Librería de Componentes UI (Design System)

PERIODO ABRIL - AGOSTO 2026



Actividades a desarrollar en la Unidad 4

Actividad # 8: Desarrollo del Backend y Base de Datos (API REST)

Objetivo de la tarea

- Construir la arquitectura del lado del servidor (Backend) y la base de datos del Sistema de Gestión de Incidentes, aplicando tecnologías de nivel industrial (Node.js o Java Spring Boot) para exponer una API RESTful escalable y segura.

1. INTRODUCCIÓN

En la actualidad las organizaciones dependen de sistemas informáticos para gestionar procesos críticos. Los sistemas Help Desk permiten registrar, administrar y dar seguimiento a incidentes tecnológicos, optimizando los tiempos de respuesta y mejorando la calidad del soporte técnico (Fielding, 2000).

El backend desarrollado utiliza Node.js, Express y PostgreSQL para implementar una API REST bajo arquitectura MVC, permitiendo la gestión de tickets y autenticación básica de usuarios. La comunicación se realiza mediante JSON y protocolos HTTP, mientras que las pruebas funcionales fueron realizadas con Postman. Node.js, s.f.; Express.js, s.f.; PostgreSQL Global Development Group, s.f.

2. PLANTEAMIENTO DEL PROBLEMA

La ausencia de un sistema centralizado para registrar incidentes dificulta el seguimiento, genera duplicidad de información y afecta la eficiencia del soporte técnico. Por ello se desarrolló una API REST con operaciones CRUD y almacenamiento persistente en PostgreSQL.

3. JUSTIFICACIÓN

El proyecto integra conocimientos de programación backend, bases de datos, arquitectura MVC, servicios REST, control de versiones y pruebas funcionales, empleando tecnologías utilizadas en entornos profesionales.

4. OBJETIVOS

4.1 OBJETIVO GENERAL

Desarrollar el backend del Sistema Help Desk mediante Node.js, Express y PostgreSQL implementando una API REST con autenticación básica.



4.2 OBJETIVOS ESPECÍFICOS

- Diseñar la base de datos en PostgreSQL.
- Implementar el CRUD de tickets.
- Implementar autenticación básica.
- Validar mediante Postman.
- Aplicar arquitectura MVC y Git/GitHub.

MARCO TEÓRICO

1. ARQUITECTURA CLIENTE-SERVIDOR

La arquitectura cliente-servidor es un modelo de comunicación en el que un cliente solicita servicios y un servidor los procesa y devuelve una respuesta. Este modelo permite separar las responsabilidades entre la interfaz de usuario y la lógica de negocio, facilitando el mantenimiento, la escalabilidad y la administración de aplicaciones distribuidas. En el presente proyecto, el navegador y Postman actuaron como clientes, mientras que Node.js y Express constituyeron el servidor encargado de procesar las solicitudes HTTP. Fielding, 2000.

2. BACKEND

El backend corresponde al conjunto de componentes que ejecutan la lógica de negocio, administran la información y gestionan el acceso a la base de datos. A diferencia del frontend, el backend no interactúa directamente con el usuario, sino que procesa las solicitudes recibidas y devuelve respuestas estructuradas. En esta actividad se desarrolló un backend utilizando Node.js y Express para gestionar tickets de soporte técnico.

3. API REST

Una API REST permite la comunicación entre aplicaciones utilizando el protocolo HTTP y recursos identificados mediante URL. Los métodos GET, POST, PUT y DELETE permiten consultar, crear, actualizar y eliminar información respectivamente. Durante el desarrollo del proyecto se implementó una API REST para administrar los tickets registrados en el sistema Help Desk. Fielding, 2000.

4. ARQUITECTURA MVC

El patrón Modelo–Vista–Controlador divide una aplicación en tres componentes



principales: el Modelo administra los datos, el Controlador procesa la lógica y las solicitudes, mientras que la Vista presenta la información al usuario. Aunque en esta actividad se desarrolló únicamente el backend, se utilizó esta arquitectura para organizar el código en modelos, controladores y rutas, facilitando su mantenimiento.

5. NODE.JS

Node.js es un entorno de ejecución basado en JavaScript que permite desarrollar aplicaciones del lado del servidor. Su arquitectura orientada a eventos y operaciones de entrada/salida no bloqueantes favorece la construcción de aplicaciones escalables y con buen rendimiento. Estas características justificaron su selección para el desarrollo de la API REST del proyecto. Node.js, s.f.

6. EXPRESS.JS

Express.js es un framework para Node.js que simplifica la creación de aplicaciones web y servicios REST. Proporciona mecanismos para definir rutas, procesar solicitudes HTTP, incorporar middleware y estructurar aplicaciones de manera modular. En el proyecto se utilizó para implementar los endpoints de la API y organizar el código siguiendo la arquitectura MVC. Express.js, s.f.

7. POSTGRESQL

PostgreSQL es un sistema gestor de bases de datos relacional de código abierto reconocido por su estabilidad, seguridad y cumplimiento de estándares SQL. Permite almacenar información de manera estructurada, garantizar la integridad de los datos y ejecutar consultas de alta complejidad. En el proyecto fue utilizado para almacenar la información de los tickets y los usuarios del sistema, constituyendo la capa de persistencia de la aplicación. PostgreSQL Global Development Group, s.f.

8. OPERACIONES CRUD

CRUD corresponde a las operaciones Create, Read, Update y Delete, fundamentales en el desarrollo de aplicaciones que administran información. En la API desarrollada estas operaciones fueron implementadas mediante los métodos HTTP POST, GET, PUT y DELETE, permitiendo registrar, consultar, actualizar y eliminar tickets del sistema Help Desk.



9. PROTOCOLO HTTP

HTTP es el protocolo utilizado para la comunicación entre clientes y servidores en aplicaciones web. Cada solicitud enviada por un cliente genera una respuesta del servidor acompañada de un código de estado. En esta actividad se emplearon principalmente los códigos 200 (OK), 201 (Created), 401 (Unauthorized), 404 (Not Found) y 500 (Internal Server Error), permitiendo informar el resultado de cada operación realizada.

10. FORMATO JSON

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos ampliamente utilizado en servicios REST debido a su simplicidad y facilidad de lectura. La API desarrollada utiliza JSON tanto para recibir información desde el cliente como para responder las solicitudes realizadas mediante Postman y otras aplicaciones compatibles.

11. POSTMAN

Postman es una herramienta que permite diseñar, probar y documentar servicios web. Facilita el envío de solicitudes HTTP utilizando diferentes métodos y posibilita visualizar las respuestas del servidor en distintos formatos. Durante el desarrollo del proyecto fue empleada para validar cada uno de los endpoints implementados y comprobar la correcta comunicación con PostgreSQL.

12. GIT

Git es un sistema distribuido de control de versiones que permite registrar los cambios realizados sobre un proyecto, mantener un historial de modificaciones y facilitar el trabajo organizado mediante ramas. En esta actividad se utilizó para administrar el código fuente del backend y mantener un flujo de trabajo basado en ramas de desarrollo.

13. GITHUB

GitHub es una plataforma de alojamiento de repositorios Git que facilita el almacenamiento del código fuente, la colaboración entre desarrolladores y el control de versiones. El proyecto fue organizado utilizando ramas específicas para cada actividad, permitiendo mantener un historial claro de los cambios realizados y facilitando la integración del trabajo desarrollado.



DESARROLLO DE LA ACTIVIDAD

1. PREPARACIÓN DEL ENTORNO DE DESARROLLO

Como etapa inicial se instaló y verificó el funcionamiento de Node.js, PostgreSQL, pgAdmin 4, Visual Studio Code y Postman. Posteriormente se creó la estructura principal del proyecto separando el frontend, backend, base de datos y documentación. Esta organización permitió desarrollar el sistema de manera ordenada y facilitar el mantenimiento del código.

2. CREACIÓN DEL PROYECTO BACKEND

Se inicializó el proyecto utilizando npm y posteriormente se instalaron las dependencias Express, pg, cors y Nodemon. Estas herramientas permitieron construir el servidor, establecer la conexión con PostgreSQL y automatizar el reinicio del servidor durante el desarrollo.

3. ORGANIZACIÓN DE LA ARQUITECTURA MVC

El backend fue estructurado siguiendo el patrón Modelo–Vista–Controlador. Se crearon carpetas independientes para la configuración de la base de datos, modelos, controladores y rutas, separando la lógica de acceso a datos de la lógica de negocio y del manejo de solicitudes HTTP.

4. CONFIGURACIÓN DE POSTGRESQL

Se creó la base de datos denominada helpdesk_db y posteriormente las tablas tickets y usuarios. La conexión entre Node.js y PostgreSQL fue implementada mediante el módulo pg, utilizando un Pool de conexiones para optimizar el acceso a la base de datos.

5. IMPLEMENTACIÓN DEL SERVIDOR

Se desarrolló el archivo principal server.js, encargado de configurar Express, habilitar el middleware cors, procesar datos JSON y registrar las rutas correspondientes a la API REST. Finalmente se configuró el puerto 3000 para recibir las solicitudes del cliente.

6. UBICACIÓN DE LAS EVIDENCIAS



Figura 1. Creación de la base de datos helpdesk_db y de la tabla tickets.

helpdesk_db/postgres@PostgreSQL 14*

helpdesk_db/postgres@PostgreSQL 14

Query Query History

1 SELECT * FROM tickets;

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	id [PK] integer	titulo character varying (150)	descripcion text	categoria character varying (20)	prioridad character varying (10)	estado
1	2	Error en disco duro	El servidor presenta fallos de lectura en el disco principal.	Hardware	Media	En Pro
2	4	Pérdida de conexión WiFi	Los usuarios reportan interrupciones frecuentes en la red inalámbr...	Red	Baja	Abiert
3	5	Cambio de memoria RAM	Se requiere reemplazar un módulo de memoria defectuoso.	Hardware	Media	En Pro
4	7	Impresora fuera de servicio	La impresora del departamento de soporte no responde.	Hardware	Media	Abiert
5	1	Servidor principal actualiza...	Se reemplazó el switch principal del Data Center.	Red	Media	En Pro

Total rows: 5 Query complete 00:00:00.562 CRLF Ln 1, Col 23

Figura 2. Prueba de conexión con PostgreSQL.

localhost:3000

localhost:3000

Pregunta a Gemini

Dar formato al texto

```
{ "mensaje": "API Help Desk funcionando correctamente" }
```

C:\WINDOWS\system32\cmd

Directorio: C:\Users\Admin\Desktop\UTM\SS\DESARROLLO DE SISTEMAS INFORMATICOS\Actividad1\ACTIVIDAD_7_BACKEND_API\backend

Mode	LastWriteTime	Length	Name
d----	30/7/2026 16:51		config
d----	30/7/2026 16:51		controllers
d----	30/7/2026 16:52		models
d----	30/7/2026 16:32		node_modules
d----	30/7/2026 16:53		routes
-a----	30/7/2026 16:32	48508	package-lock.json
-a----	30/7/2026 16:33	498	package.json
-a----	30/7/2026 17:19	415	server.js

PS C:\Users\Admin\Desktop\UTM\SS\DESARROLLO DE SISTEMAS INFORMATICOS\Actividad1\ACTIVIDAD_7_BACKEND_API\backend> npm run dev

> backend@1.0.0 dev

> nodemon server.js

[nodemon] 3.1.14

[nodemon] to restart at any time, enter 'rs'

[nodemon] watching path(s): *.*

[nodemon] watching extensions: js,mjs,cjs,json

[nodemon] starting 'node server.js'

Servidor ejecutándose en http://localhost:3000

ANÁLISIS DE LA ETAPA DESARROLLADA

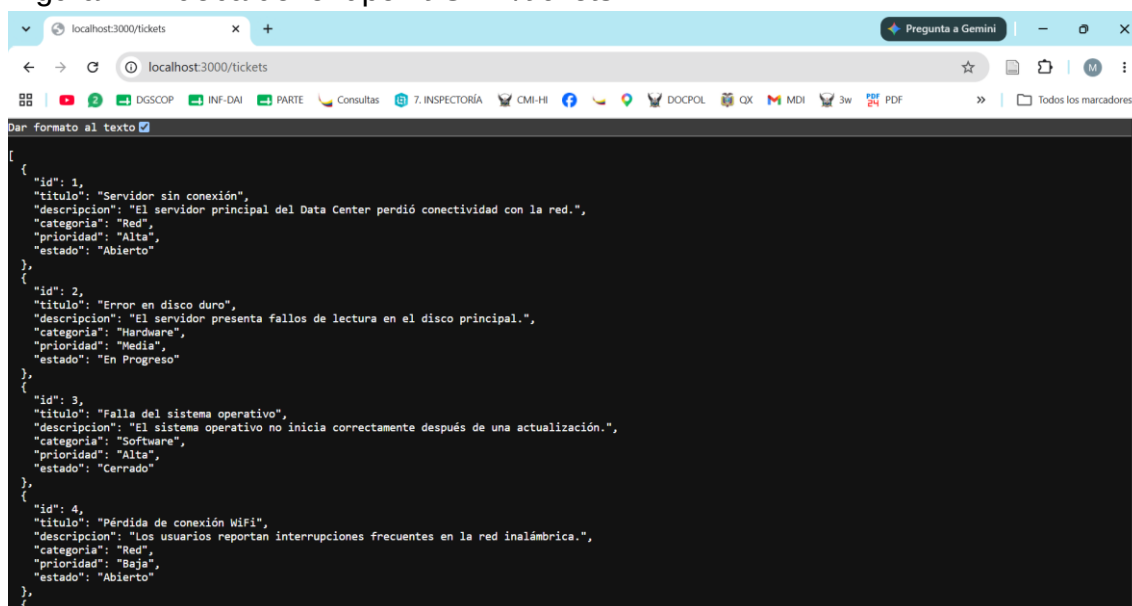
La correcta configuración del entorno de desarrollo permitió establecer una comunicación estable entre la aplicación Node.js y PostgreSQL. La organización del proyecto bajo arquitectura MVC facilitó la separación de responsabilidades y proporcionó una base sólida para la implementación posterior de la API REST y de los mecanismos de autenticación.

Implementación de la API REST (GET y POST)

1. DESARROLLO DEL ENDPOINT GET /TICKETS

El primer endpoint implementado permitió consultar la totalidad de los tickets almacenados en la base de datos PostgreSQL. La solicitud fue desarrollada utilizando el método HTTP GET y devuelve un arreglo de objetos en formato JSON. Para su implementación se definieron las rutas correspondientes, el controlador encargado de procesar la petición y el modelo responsable de ejecutar la consulta SQL. Durante las pruebas realizadas con Postman se verificó la correcta recuperación de los registros almacenados.

Figura 4. Prueba del endpoint GET /tickets.

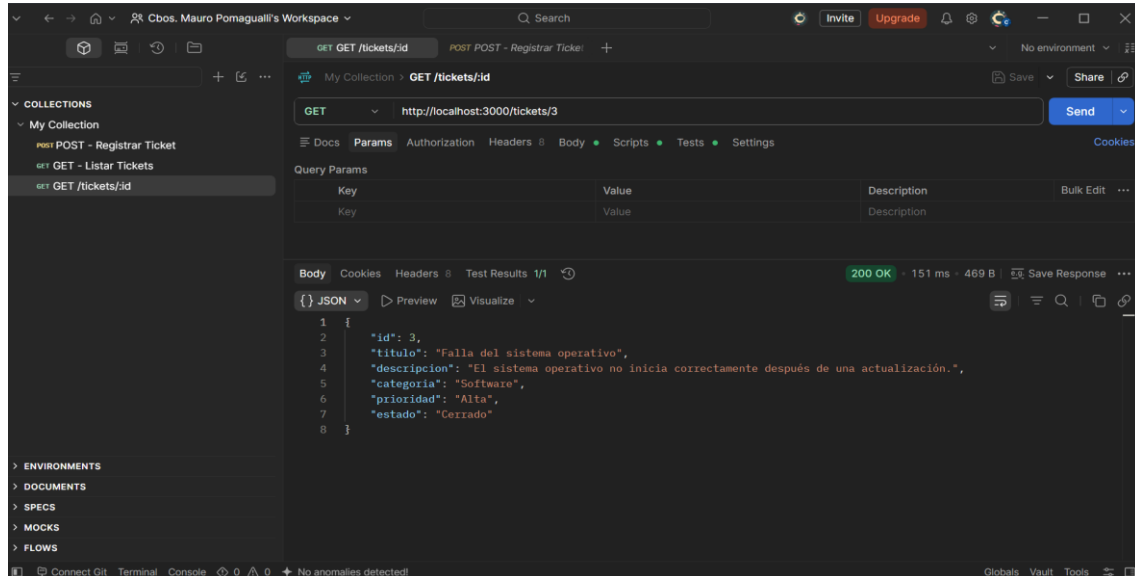


2. DESARROLLO DEL ENDPOINT GET /TICKETS/{ID}

Posteriormente se implementó el endpoint destinado a consultar un ticket específico mediante su identificador. La solicitud recibe el parámetro id desde la URL, ejecuta una consulta SQL parametrizada y devuelve únicamente el registro solicitado. En caso de no existir el identificador, la API responde con el código

HTTP correspondiente y un mensaje informativo en formato JSON.

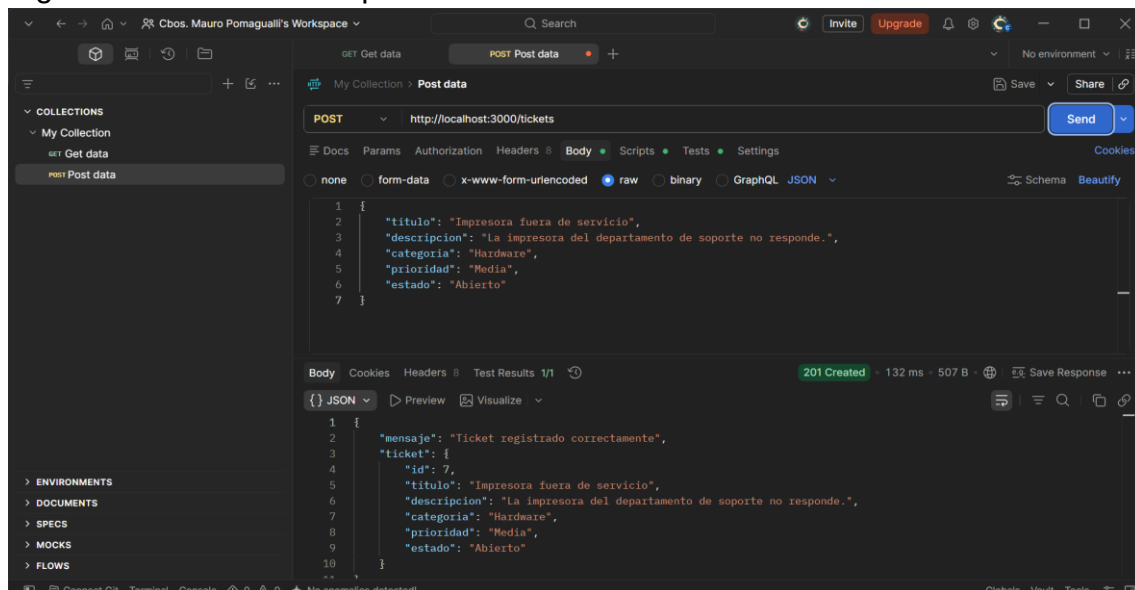
Figura 5. Prueba del endpoint GET /tickets/{id}.



3. DESARROLLO DEL ENDPOINT POST /TICKETS

El método POST fue desarrollado para registrar nuevos incidentes dentro del sistema Help Desk. La información enviada por el cliente mediante formato JSON es recibida por Express, validada por el controlador y posteriormente almacenada en PostgreSQL mediante una sentencia INSERT. Una vez finalizado el proceso, la API devuelve el código HTTP 201 (Created) junto con el registro recién creado.

Figura 6. Prueba del endpoint POST /tickets.



4. ANÁLISIS DE LOS RESULTADOS

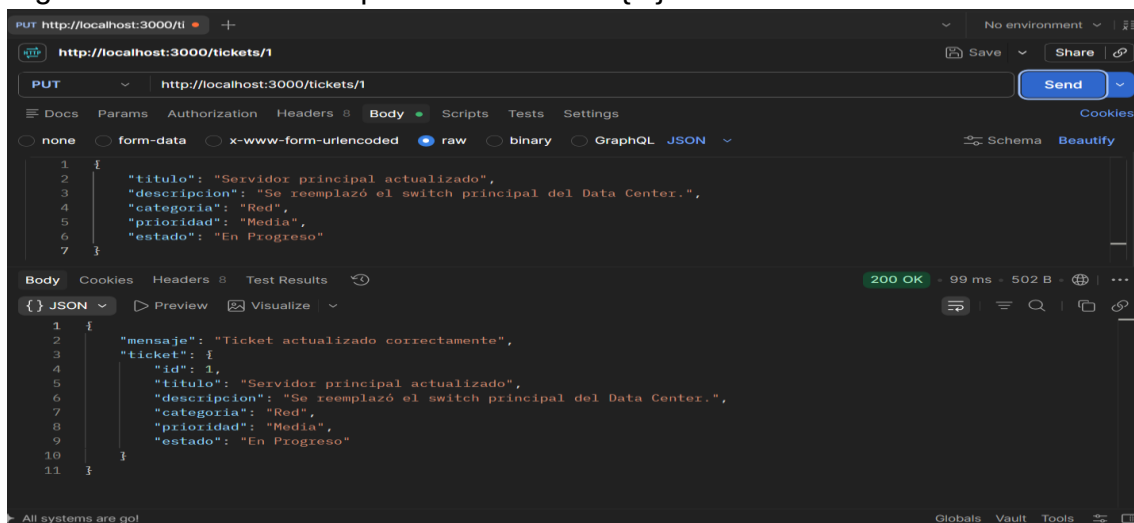
Las pruebas efectuadas con Postman demostraron el correcto funcionamiento de los tres primeros endpoints implementados. La API respondió utilizando mensajes en formato JSON y códigos HTTP apropiados, verificándose además la correcta comunicación entre Node.js, Express y PostgreSQL. Estas pruebas permitieron confirmar el adecuado funcionamiento de la arquitectura MVC implementada durante el desarrollo del proyecto.

IMPLEMENTACIÓN DE LA API REST (PUT, DELETE Y AUTENTICACIÓN)

1. DESARROLLO DEL ENDPOINT PUT /TICKETS/{ID}

El endpoint PUT fue implementado para actualizar la información de un ticket existente. La solicitud recibe el identificador del ticket mediante la URL y los nuevos datos en formato JSON. El controlador procesa la petición, valida la información recibida y ejecuta una sentencia UPDATE sobre PostgreSQL. Cuando la operación concluye correctamente, la API devuelve el código HTTP 200 junto con el registro actualizado, confirmando que la modificación fue almacenada satisfactoriamente.

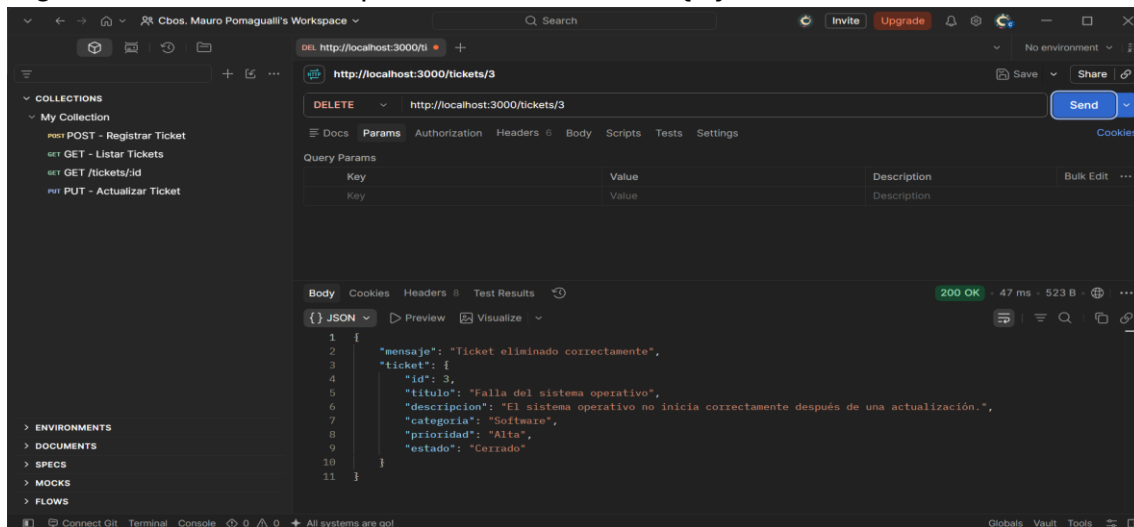
Figura 7. Prueba del endpoint PUT /tickets/{id}.



2. DESARROLLO DEL ENDPOINT DELETE /TICKETS/{ID}

Para completar el CRUD se implementó el método DELETE, encargado de eliminar un ticket de la base de datos mediante su identificador. El controlador ejecuta una sentencia DELETE parametrizada y devuelve un mensaje confirmando la eliminación del registro. Si el identificador no existe, la API responde con el código HTTP correspondiente y un mensaje descriptivo en formato JSON.

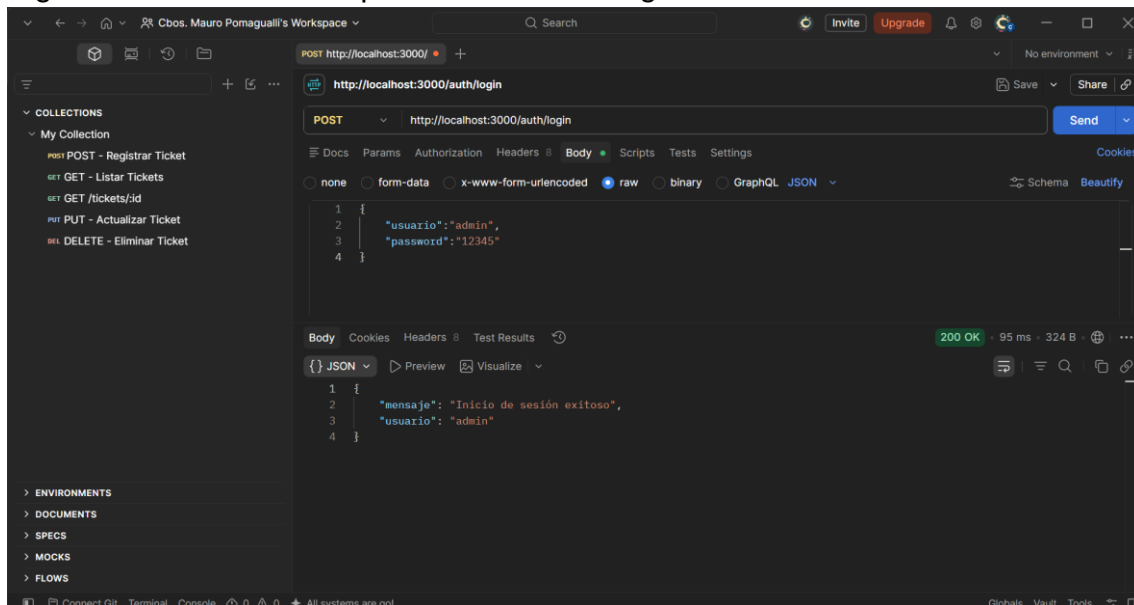
Figura 8. Prueba del endpoint DELETE /tickets/{id}.

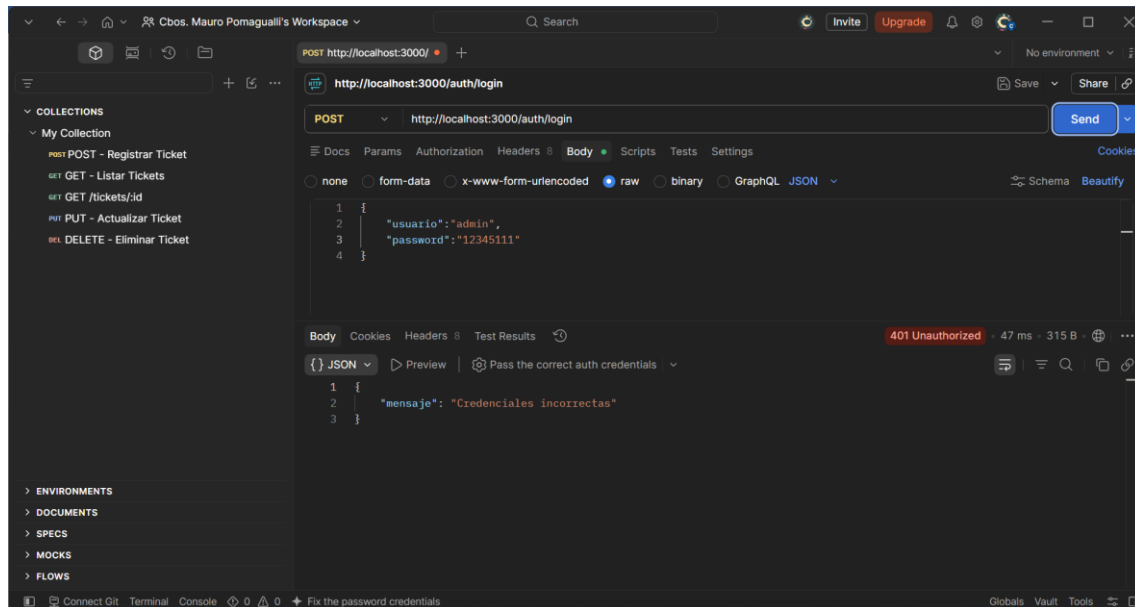


3. IMPLEMENTACIÓN DE LA AUTENTICACIÓN

Como complemento al backend se desarrolló un mecanismo básico de autenticación utilizando una tabla de usuarios almacenada en PostgreSQL. Se implementó el endpoint POST /auth/login, el cual recibe el nombre de usuario y la contraseña mediante formato JSON. El modelo realiza la consulta a la base de datos y el controlador valida las credenciales, devolviendo una respuesta satisfactoria cuando la autenticación es correcta o un mensaje de error cuando las credenciales no coinciden.

Figura 9. Prueba del endpoint POST /auth/login.





4. ANÁLISIS DE LOS RESULTADOS

Las pruebas realizadas permitieron verificar que las operaciones de actualización, eliminación y autenticación funcionan correctamente. La comunicación entre Express y PostgreSQL respondió de forma estable durante todas las pruebas efectuadas en Postman, devolviendo respuestas en formato JSON y códigos HTTP acordes con cada operación. Con estas implementaciones se completó el ciclo CRUD requerido por la actividad y se incorporó un mecanismo básico de validación de usuarios.

EVIDENCIAS DE CREACION DE LA RAMA “feature/backend-api”

Link GIT: https://github.com/mpomagualli1743-commits/ACTIVIDAD_1.git

Link Drive de los archivos: <https://drive.google.com/drive/folders/1KsNQ-FZODLfDTOR7zldCIDKjFMnDW-7A?usp=sharing>

Figura 10. Creación de la rama feature/backend-api

```
MINGW64/c/Users/Admin/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1...
develop
feature/maquetacion-html
feature/responsive-layout
feature/ui-kit
main

dmin@DESKTOP-F80PBTH MINGW64 ~/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1
ACTIVIDAD_7_BACKEND_API (develop)
git checkout develop
Already on 'develop'
Your branch is up to date with 'origin/develop'.

dmin@DESKTOP-F80PBTH MINGW64 ~/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1
ACTIVIDAD_7_BACKEND_API (develop)
git pull origin develop
From https://github.com/mpomagualli1743-commits/ACTIVIDAD_1
* branch      develop      -> FETCH_HEAD
Already up to date.

dmin@DESKTOP-F80PBTH MINGW64 ~/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1
ACTIVIDAD_7_BACKEND_API (develop)
git checkout -b feature/backend-api
Switched to a new branch 'feature/backend-api'

dmin@DESKTOP-F80PBTH MINGW64 ~/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1
ACTIVIDAD_7_BACKEND_API (feature/backend-api)
git branch
  develop
  feature/backend-api
  feature/maquetacion-html
  feature/responsive-layout
  feature/ui-kit
  main

dmin@DESKTOP-F80PBTH MINGW64 ~/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1
ACTIVIDAD_7_BACKEND_API (feature/backend-api)
git branch
  develop
  feature/backend-api
  feature/maquetacion-html
  feature/responsive-layout
  feature/ui-kit
  main

dmin@DESKTOP-F80PBTH MINGW64 ~/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1
ACTIVIDAD_7_BACKEND_API (feature/backend-api)
git push -u origin feature/backend-api
remote: Permission to mpomagualli1743-commits/ACTIVIDAD_1.git denied to Extronaz.
fatal: unable to access 'https://github.com/mpomagualli1743-commits/ACTIVIDAD_1.git/': The requested URL returned error: 403
```

Figura 11. Push de la actividad 7

```
MINGW64/c/Users/Admin/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1...

Admin@DESKTOP-F80PBTH MINGW64 ~/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1
/ACTIVIDAD_7_BACKEND_API (feature/backend-api)
$ git status --untracked-files=all
On branch feature/backend-api
Your branch is ahead of 'origin/feature/backend-api' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

Admin@DESKTOP-F80PBTH MINGW64 ~/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1
/ACTIVIDAD_7_BACKEND_API (feature/backend-api)
$ git push
git: 'credential-manager-core' is not a git command. See 'git --help'.
Enumerating objects: 28, done.
Counting objects: 100% (28/28), done.
Delta compression using up to 8 threads
Compressing objects: 100% (25/25), done.
Writing objects: 100% (27/27), 22.69 KiB | 1.51 MiB/s, done.
Total 27 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/mpomagualli1743-commits/ACTIVIDAD_1.git
   fcc00ff..22d5faa  feature/backend-api -> feature/backend-api

Admin@DESKTOP-F80PBTH MINGW64 ~/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1
/ACTIVIDAD_7_BACKEND_API (feature/backend-api)
$ git push -u origin feature/backend-api
git: 'credential-manager-core' is not a git command. See 'git --help'.
branch 'feature/backend-api' set up to track 'origin/feature/backend-api'.
Everything up-to-date
```

Figura 12. Fusión develop.

```
MINGW64/c/Users/Admin/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1...
Already up to date.

Admin@DESKTOP-F80PBTH MINGW64 ~/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1
/ACTIVIDAD_7_BACKEND_API (develop)
$ git merge feature/backend-api
Updating fcc00ff..22d5faa
Fast-forward
 ACTIVIDAD_7_BACKEND_API/.gitignore | 15 +
 ACTIVIDAD_7_BACKEND_API/README.md | 66 +
 ACTIVIDAD_7_BACKEND_API/backend/config/db.js | 9 +
 .../backend/controllers/authController.js | 28 +
 .../backend/controllers/ticketController.js | 129 ++
 .../backend/models/ticketModel.js | 61 +
 .../backend/models/userModel.js | 15 +
 ACTIVIDAD_7_BACKEND_API/backend/package-lock.json | 1392 ++++++
 ACTIVIDAD_7_BACKEND_API/backend/package.json | 27 +
 .../backend/routes/authRoutes.js | 5 +
 .../backend/routes/ticketRoutes.js | 35 +
 ACTIVIDAD_7_BACKEND_API/backend/server.js | 39 +
 ACTIVIDAD_7_BACKEND_API/db/helpdesk_db.sql | 73 +
 ACTIVIDAD_7_BACKEND_API/frontend/Index.html | 107 ++
 ACTIVIDAD_7_BACKEND_API/frontend/Reportar.html | 148 +++
 ACTIVIDAD_7_BACKEND_API/frontend/style.css | 468 ++++++
 ACTIVIDAD_7_BACKEND_API/frontend/tickets.html | 203 +++
17 files changed, 2820 insertions(+)
create mode 100644 ACTIVIDAD_7_BACKEND_API/.gitignore
create mode 100644 ACTIVIDAD_7_BACKEND_API/README.md
create mode 100644 ACTIVIDAD_7_BACKEND_API/backend/config/db.js
create mode 100644 ACTIVIDAD_7_BACKEND_API/backend/controllers/authController.js
create mode 100644 ACTIVIDAD_7_BACKEND_API/backend/controllers/ticketController.js
create mode 100644 ACTIVIDAD_7_BACKEND_API/backend/models/ticketModel.js
create mode 100644 ACTIVIDAD_7_BACKEND_API/backend/models/userModel.js
create mode 100644 ACTIVIDAD_7_BACKEND_API/backend/package-lock.json
create mode 100644 ACTIVIDAD_7_BACKEND_API/backend/package.json
create mode 100644 ACTIVIDAD_7_BACKEND_API/backend/routes/authRoutes.js
create mode 100644 ACTIVIDAD_7_BACKEND_API/backend/routes/ticketRoutes.js
create mode 100644 ACTIVIDAD_7_BACKEND_API/backend/server.js
create mode 100644 ACTIVIDAD_7_BACKEND_API/db/helpdesk_db.sql
create mode 100644 ACTIVIDAD_7_BACKEND_API/frontend/Index.html
create mode 100644 ACTIVIDAD_7_BACKEND_API/frontend/Reportar.html
create mode 100644 ACTIVIDAD_7_BACKEND_API/frontend/style.css
create mode 100644 ACTIVIDAD_7_BACKEND_API/frontend/tickets.html

Admin@DESKTOP-F80PBTH MINGW64 ~/Desktop/UTM/S5/DESARROLLO DE SISTEMAS INFORMATICOS/Actividad1
/ACTIVIDAD_7_BACKEND_API (develop)
$ git push origin develop
git: 'credential-manager-core' is not a git command. See 'git --help'.
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/mpomagualli1743-commits/ACTIVIDAD_1.git
 fcc00ff..22d5faa develop -> develop
```



REFERENCIAS

1. PostgreSQL Global Development Group. (s.f.). <https://www.postgresql.org/docs/>
2. Express.js. (s.f.). Express - Node.js web application framework. <https://expressjs.com/>
3. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures.
4. Node.js. (s.f.). Node.js Documentation. <https://nodejs.org/docs/latest/api/>
5. Git. (s.f.). Git Documentation. <https://git-scm.com/doc>
6. GitHub. (s.f.). GitHub Docs. <https://docs.github.com/>
7. Postman, Inc. (s.f.). Postman Learning Center. <https://learning.postman.com/>
8. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures.